

OpenFin

OpenFin · docs/latest · generated 2026-08-15

How StarUI runs inside an OpenFin workspace — and how the rest of the platform stays OpenFin-free.

The containment rule: only `@wellsfargo-starui/openfin` may import `@openfin/core` (ESLint-enforced). Everything else uses the seams below, all of which **degrade to no-ops in a plain browser**, so the same app code runs unchanged in a browser tab and inside the workspace.

Package surface

SUBPATH	CONTENTS
<code>@wellsfargo-starui/openfin</code>	the workspace shell — <code>initWorkspace</code> + its callable pieces, registry config
<code>.../host</code>	the contained seams: <code>isOpenFin</code> , identity/IAB/interop helpers, <code>OpenFinRuntime</code> , <code>openOpenFinPopout</code>
<code>.../config</code>	side-effect-free entry (safe in a plain browser / Vite dev server — <code>@openfin/workspace-platform</code> reads <code>fin</code> at module eval, so this subpath deliberately avoids it): ConfigManager wiring, IAB topics, action ids
<code>.../dock-editor</code>	dock layout editor helpers
<code>.../test-bridge</code>	<code>installTestBridge</code> — dev/e2e IAB channel, code-split out of prod

The one detector and the seams (`.../host`)

`isOpenFin()` is a bare `fin`-global presence check — THE detector; nothing else in `packages/` defines its own. The seams every other package uses:

- Identity** — `getFinMe()`, `getOpenFinWindowIdentity()` (`{uuid, name}` — unique per view; the echo-suppression source id), `getCurrentView()`, `resolveOpenFinIdentity()` (`customData > URL params > overrides > defaults`).
- IAB** — `publishIabTopic`, `subscribeIabTopic` (wildcard-source subscribe; returns a disposer), `connectIabChannel`. The only sanctioned IAB path outside `packages/openfin`.
- Interop** — `getInteropClient()`, `isInteropAvailable()` — thin views over `fin.me.interop`.
- Platform/window control** — `createPlatformView()`, `closeCurrentWindow()`.
- OpenFinRuntime** — the OpenFin implementation of the `RuntimePort` contract (`@wellsfargo-starui/core/host`): `openSurface`, `setTheme`, `onWorkspaceSave`, saved-view titles. The browser twin is `BrowserRuntime` (`@wellsfargo-starui/core/host/browser`).

Running an app under the workspace (star-demo shape)

1. **Manifest** (`public/platform/manifest.fin.json`) — platform `uuid`, `providerUrl` pointing at the app's `/platform/provider` route, `customSettings` (`appId`, `userId`, `seedConfigUrl`, `useRest`, `configServiceRestUrl`), and `fdc3InteropApi` in the default view/window options (OPTIONAL — it only enables the FDC3 *fallback* for grid linking; interop is primary).
2. **Launcher** — `npm run client` runs `launch.mjs` (`@openfin/node-adapter`): fetches the manifest, launches, and quits the platform on Ctrl-C.
3. **Provider window** (the route `providerUrl` names) calls `initWorkspace(config)` once. Apps with their own bootstrap prewire the ConfigManager first (`setConfigManager` from `.../config`) so `initWorkspace` adopts it.
4. **Views** render normal app routes; `useHostedStarui` / `StaruiIdentityProvider` bridge workspace identity into the same `<StarGrid>` code a browser tab uses. Dev identity (`'TestApp'` / `'dev1'`) is refused unless `devDefaults: true`.

`initWorkspace` and its pieces

`initWorkspace(config?: WorkspaceConfig)` boots: manifest read → `ensureConfigService` → platform scope → migrations → theme sync → dock + notifications registration → workspace-persistence override → platform init. The pieces are exported for hosts that need them separately:

- `ensureConfigService(customSettings, providerUrl, { mode })` — `'auto'` (default) constructs a ConfigManager from the manifest; `'require-rewired'` **throws** unless the host called `setConfigManager(...)` first (no silent construction).
- `runPlatformScopeMigrations()` — four idempotent, individually best-effort persisted-state healing sweeps. `WorkspaceConfig.migrations` defaults **on** — existing installs depend on the healing.

`WorkspaceConfig` highlights: `components` (dock + notifications on; **Home + Store opt-in**), `theme` (dark-ramp brand overrides), `dockIcon` (raster only), `dock.excludeTools`, `themeToggle*Icon` (presence adds the dock toggle), `configService`, `devTools` (defaults to the dev bundle; gates the devtools **menu entries**, handlers stay registered), and `customActions` — genuinely merged over the built-ins, app handler wins on id collision.

Built-in dock/tool actions (11): launch-app, launch-component (the one persisted dock configs reference), toggle-theme, open-workspace-setup, open-data-providers, open-config-browser, reload-dock, show-devtools†, inspect-shared-worker†, export-config, toggle-provider-window† († devtools-gated). Plus the view-tab right-click rename action.

Theme

`applyTheme` (from `@wellsfargo-starui/design-system/apply-theme`) is THE single theme writer — it stamps `data-theme`, `data-ag-theme-mode`, `data-variant`, `data-cvd` and persists `starui:theme` /

`starui:cvd` / `starui:variant`. Every `setTheme` path routes through it: both dock toggle flavors, the workspace boot sync, `OpenFinRuntime.setTheme`, and the inbound IAB `theme-changed` handler. Callers must spread the current state — `applyTheme({ ...getTheme(), theme })` — or the persisted cvd/variant choice is wiped. Broadcast (IAB / BroadcastChannel) stays in the runtimes; `applyTheme` itself never broadcasts.

Popouts — two mechanisms, deliberately

	URL-WINDOW FAMILY	POPOUTPORTAL / POPPABLE
What	a separate document at a route	the same React tree reparented into a detached OS window
Created by	<code>openOpenFinPopout</code> (platform <code>createWindow</code> → workspace-aware, dockable, snapshot-saved); <code>openChildToolWindow</code> adds the manifest-origin + inspectable-menu wrapper for the provider window; <code>RuntimePort.openSurface</code> ; <code>toolSurfaces</code>	<code>window.open</code> / <code>openFinWindowOpener</code>
Semantics	named-window dedup, focus-on-reopen, navigate-on-state-URL; survives parent reload	live shared state (context, instant updates); dies with the parent
Use for	tool windows (provider editor, config browser), app views	toolbar/panel pop-outs where shared state matters

`toolSurfaces` (`@wellsfargo-starui/core/host`) is the ONE definition of the platform tool windows: `openProviderEditorSurface(runtime, { providerId? })` (`/dataproviders`, 1180×760) and `openConfigBrowserSurface(runtime)` (`/config-browser`, 1100×720) — same name/route/size triples the dock handlers use, working over either runtime.

Workspace save/restore

The `ConfigService` is the **single source of truth** — one `appConfig` row per workspace (`WS_<id>`, `componentType: 'workspace'`) holding the OpenFin snapshot; OpenFin's own IndexedDB is deliberately not a fallback.

Grid state is flushed **before** the snapshot is captured by a two-part handshake:

1. An awaited channel fan-out (`marketsui-workspace-save-channel`): the platform dispatches `workspace-saving` to every connected view and awaits them; views register via `useWorkspaceSaveEvent(saveCb)`.
2. The `RuntimePort.onWorkspaceSave` seam (fire-and-forget `workspace-saved` bridge) for after-the-fact listeners.

`<StarGrid>` wires the flush automatically — and also flushes on `beforeunload`, `pagehide`, `view destroyed`, and `unmount`, because workspace drag/move does **not** fire `workspace-saving`. Restored views keep per-view state via `advanced.instanceId` (view `customData`).

Grid linking

`contextLink={{ enabled, mode }}` on `<StarGrid>` — `mode: 'rowId' | 'fields'`. The transport facade is `GridLinkTransport` (`current`, `addContextListener`, `broadcast`); **interop is primary** (`useInteropChannel` — the dock Link button joins interop context groups that `window.fdc3` does not reliably reflect), FDC3 is the fallback (needs `fdc3InteropApi` in the manifest). Enabling linking with neither available logs a **loud console error** naming the fix. Wire format: `starui.gridSelection` contexts with per-view echo suppression. Deep dive: `docs/OPENFIN_GRID_LINKING.md`.

Testing

- `apps/e2e-openfin/` — Playwright + `@openfin/node-adapter` drives the real workspace through the `marketsui-test-bridge` IAB channel (installed by the provider window in dev / `?e2eBridge=1`).
- OpenFin flows that can't run headlessly (dock, save/restore, linking) carry a manual-validation backlog in `SIMPLIFICATION_ROADMAP.md` (Phase 5).